

Adv. Database Material First Two Modules

1. What is DBMS

A **Database Management System (DBMS)** is specialized system software designed to create, maintain, search, and manage structured databases. It provides a standardized interface between users, application programs, and physical database storage.

Core Database Evolution Waves

- **1st Wave (1960s-1970s):** Hierarchical (tree structure) & Network (graph structure) models.
- **2nd Wave (1970s-1980s):** Relational Model (tables with keys).
- **3rd Wave (1990s):** Object-Oriented DBMS (OODBMS) & Object-Relational (ORDBMS)[cite: 2, 3].
- **4th Wave (2000s+):** NoSQL, Graph, and Multidimensional databases.

DBMS vs RDBMS Comparison

Feature	DBMS	RDBMS
Data Storage	Files or flat structures	Tabular relations (rows & columns)
Relationships	Not supported or primitive	Enforced via Primary Key/Foreign Key links
Architecture	Single-user / File-based	Supports multi-user Client-Server setup
Tools	FoxPro, DBase	Oracle, MySQL, MS SQL Server

Architecture Types

- **2-Tier Architecture:** Client directly connects to the Database Server (Client holds presentation + business logic).
- **3-Tier Architecture:** Client interacts with an **Application Server** (Middle Tier), which enforces business rules and handles database access.

2. RELATIONAL MODEL

Proposed by Edgar F. Codd in 1970, the **Relational Model** organizes data into logical two-dimensional tables called **Relations**.

Relation: STUDENT

Key Terminology

- **Tuple:** A single horizontal row in a relation representing a record[cite: 6, 16].
- **Attribute:** A named vertical column representing a field/property[cite: 6, 16].
- **Domain:** The set of permitted atomic values for an attribute.
- **Primary Key (PK):** An attribute that uniquely identifies each tuple (No NULLs, No Duplicates).
- **Candidate Key:** Any set of attributes capable of uniquely identifying a row.
- **Foreign Key (FK):** An attribute referencing the Primary Key of another table to link records.

3. Functional Dependency (FD)

A **Functional Dependency** describes a structural relationship between attributes, written as $X \rightarrow Y$ ("X determines Y"). It means that for any valid state of a database, if two tuples agree on attribute X , they must agree on attribute Y .

Types of Functional Dependency

1. **Full Functional Dependency:** An attribute depends on the *entire* composite primary key, not just a part of it.
 - *Example:* $\{Student_ID, Course_ID\} \rightarrow Final_Grade$.
2. **Partial Dependency:** A non-key attribute depends on only *part* of a composite primary key.
 - *Example:* $\{Student_ID, Course_ID\} \rightarrow Student_Name$
(depends only on $Student_ID$).
3. **Transitive Dependency:** A non-key attribute depends on another non-key attribute, forming a chain: $A \rightarrow B \rightarrow C$.
 - *Example:* $Course_ID \rightarrow Instructor \rightarrow Instructor_Office$ [cite: 21].

4. NORMALIZATION

Normalization is a step-by-step database design process used to decompose complex, redundant tables into smaller, well-structured relations to eliminate insertion, update, and deletion anomalies.

Normal Forms Breakdown

- **First Normal Form (1NF):**
 - Rule 1: Every cell must hold a single, indivisible (atomic) value.
 - Rule 2: Eliminate repeating groups/columns [cite: 23].
- **Second Normal Form (2NF):**
 - Rule 1: Must be in 1NF [cite: 23].
 - Rule 2: All non-key attributes must be fully functionally dependent on the primary key (no partial dependencies).
- **Third Normal Form (3NF):**

- Rule 1: Must be in 2NF[cite: 25].
- Rule 2: No non-key attribute should depend on another non-key attribute (no transitive dependencies).

5. WHAT IS STRUCTURED QUERY LANGUAGE (SQL)

SQL is a standardized, non-procedural programming language used to define, query, manipulate, and control data stored inside relational databases.

Classification of SQL Commands

- **DDL (Data Definition Language):** Defines table schemas and database structures (CREATE, ALTER, DROP, TRUNCATE, RENAME)[cite: 26, 27, 31].
- **DML (Data Manipulation Language):** Handles data extraction and row modification (SELECT, INSERT, UPDATE, DELETE)[cite: 26].
- **TCL (Transaction Control Language):** Manages database transaction state changes (COMMIT, ROLLBACK, SAVEPOINT).
- **DCL (Data Control Language):** Manages user privileges (GRANT, REVOKE).

6. JOINS it's Types

A **JOIN** operation combines rows from two or more tables based on a related common column (typically a Foreign Key linking to a Primary Key).

Matches only All Left + Matches All Right + Matches

Types of Joins

1. **Inner Join (Equi Join):** Returns only tuples that have matching values in both tables[cite: 35].

SQL

```
SELECT e.Ename, d.Dname
```

```
FROM EMP e INNER JOIN DEPT d ON e.Deptno = d.Deptno;[cite: 36]
```

2. **Left Outer Join:** Returns all records from the left table, along with matching records from the right table (unmatched right fields show NULL)[cite: 37].

SQL

```
SELECT e.Ename, d.Dname
```

```
FROM EMP e, DEPT d WHERE e.Deptno = d.Deptno(+);[cite: 37]
```

3. **Right Outer Join:** Returns all records from the right table, along with matching records from the left table[cite: 38].

SQL

```
SELECT e.Ename, d.Dname
```

```
FROM EMP e, DEPT d WHERE e.Deptno(+) = d.Deptno;[cite: 38]
```

4. **Self Join:** Joining a table to itself using aliases to compare rows within the same table.

SQL

```
SELECT e.Fname AS Employee, m.Fname AS Manager
```

```
FROM emp_mstr e, emp_mstr m WHERE e.mngrno = m.empno;[cite: 40]
```

7. SUBQUERY

A **Subquery** (or Inner Query) is a SELECT statement nested inside an outer main query's WHERE, HAVING, or FROM clause.

Subquery Categories

- **Single-Row Subquery:** Returns exactly one row. Uses single-row operators (=, >, <, >=, <=)[cite: 41, 42].

SQL

```
SELECT Ename FROM EMP
```

```
WHERE Salary > (SELECT Salary FROM EMP WHERE Ename = 'Peter');[cite: 41]
```

- **Multiple-Row Subquery:** Returns multiple rows. Uses set operators (IN, ANY, ALL).

SQL

```
SELECT Ename, Salary FROM EMP
```

```
WHERE Salary IN (SELECT MIN(Salary) FROM EMP GROUP BY Deptno);[cite: 42]
```

8. VIEW

A **VIEW** is a virtual table that does not physically store data on disk; it stores an SQL query definition that executes dynamically when queried.

Types of Views

1. **Updatable View:** Simple views built on a single table without aggregate operations. Allows INSERT, UPDATE, and DELETE operations that pass directly to the base table.
2. **Read-Only View:** Created using the WITH READ ONLY clause[cite: 43]. Prevents all data modifications[cite: 43].

SQL

```
CREATE VIEW EMP80 AS
```

```
SELECT EmpID, EmpName, Salary
```

```
FROM EMP WHERE DeptID = 10;[cite: 44]
```

9. PL/SQL Function and Procedure

PL/SQL Functions and Procedures are named, reusable code blocks stored directly in compiled form inside the database.

Comparison & Syntax

- **Function Syntax:**

SQL

```
CREATE OR REPLACE FUNCTION empchk(dno IN NUMBER)
```

```
RETURN VARCHAR2 IS
```

```
    enm VARCHAR2(10);
```

```
BEGIN
```

```
    SELECT ename INTO enm FROM emp WHERE deptno = dno;
```

```
    RETURN enm;
```

```
END;
```

- **Key Advantages:** Code reusability, reduced network traffic, optimized disk I/O, and secure execution[cite: 45].

10. TRIGGER

A **Database Trigger** is a stored PL/SQL procedure that automatically executes ("fires") when a specific event (like INSERT, UPDATE, or DELETE) occurs on a table or view.

Key Trigger Concepts

- **Row-Level Trigger (FOR EACH ROW):** Executes once for *every individual row* modified[cite: 46, 47].
- **Statement-Level Trigger:** Default trigger mode; executes *once per statement*, regardless of affected rows.
- **Timing:** BEFORE (executes prior to saving changes) or AFTER (executes following table modification).
- **Pseudo-Variables:** :OLD stores pre-change data values; :NEW stores post-change values.

11. TRANSACTION MANAGEMENT

A **Transaction** is a logical unit of database work consisting of one or more sequential SQL operations (Reads and Writes).

Transaction States

1. **Active:** Initial state where execution begins with Read/Write operations.
2. **Partially Committed:** Final operations are processed in memory (RAM), but changes are not yet written to disk.
3. **Committed:** Changes are permanently saved to physical storage[cite: 48, 49, 50].
4. **Failed:** Transaction encounters an error or hardware interruption[cite: 49, 50].
5. **Aborted:** Changes are undone (ROLLBACK) and the database reverts to its prior state[cite: 49, 50].

12. ACID Properties

ACID properties guarantee database reliability during concurrent access and system crashes.

A - Atomicity → "All or Nothing" execution

C - Consistency → Preserves database validity rules[cite: 50, 51]

I - Isolation → Concurrent transactions do not interfere[cite: 50, 51]

D - Durability → Committed updates survive system crashes[cite: 50, 51]

- **Atomicity:** Ensures all operations complete successfully; otherwise, the transaction is completely aborted[cite: 50].
- **Consistency:** Keeps database state valid according to defined integrity constraints before and after execution.
- **Isolation:** Ensures concurrently executing transactions remain invisible to one another until committed.
- **Durability:** Guarantees committed updates are permanently recorded on disk and persist even after system failure.

13. TCL – TRANSACTION CONTROL LANGUAGE

Transaction Control Language (TCL) commands manage changes introduced by DML statements and ensure data consistency.

COMMIT: Permanently saves all transaction updates to disk.

- **ROLLBACK:** Undoes all pending changes made within the current transaction[cite: 52].
- **SAVEPOINT:** Sets a intermediate checkpoint inside a transaction for selective rollbacks (ROLLBACK TO SP_NAME)[cite: 53].

14. What is a Deadlock

A **Deadlock** is an execution blockage where two or more concurrent transactions are stuck in a circular wait condition, each holding a lock on a resource that the other requires.

The Four Coffman Conditions

For a deadlock to occur, all four conditions must hold simultaneously:

1. **Mutual Exclusion:** Resources cannot be shared simultaneously.
2. **Hold and Wait:** A process holds one resource while waiting for another.
3. **No Preemption:** Locks cannot be forcibly taken away from a transaction.
4. **Circular Wait:** Transaction \$T_1\$ waits for \$T_2\$, and \$T_2\$ waits for \$T_1\$.

Resolution Strategies

- **Deadlock Prevention:** Protocols order resource requests sequentially to prevent circular wait states.
- **Deadlock Recovery:** The DBMS detects the deadlock loop, selects a "victim" transaction, and aborts/rolls it back to free its locks.